SSH from Windows with a PIV smart card

Companion to the SSSD Smart Card Profile Manager. This is the Windows half: how to log in over SSH to a host that the manager has already configured, using the certificate on a PIV smart card or USB token.

All names, thumbprints and GUIDs below are examples. Substitute your own.

---

1. What the server already does for you

Nothing has to be copied to the server. No `authorized_keys` file is edited, no public key is exported from the card, and no per-user setup is needed beyond what the manager already applied.

The chain on the Linux side is:

- The card holds an X.509 certificate issued by your CA, for example by `EXAMPLE-Sub-CA`, with the user's UPN (`someone@ad.example.com`) in the subject alternative name.
- SSSD maps that certificate to a user account using its certificate matching and mapping rules, and **derives the SSH public key directly from the public key inside the certificate**.
- `sshd` asks SSSD for that key at login time, through the directives below.

```
AuthorizedKeysCommand /usr/bin/sss_ssh_authorizedkeys
AuthorizedKeysCommandUser root
```

So the key the server will accept is a function of the certificate on the card.

Two consequences worth stating plainly:

- **Revocation covers SSH too.** Revoke the certificate, and once SSSD honours the CRL the derived key stops being returned by `sss_ssh_authorizedkeys` — the SSH login dies with it. There is no stale `authorized_keys` entry left behind to clean up. This is the main reason to do SSH this way rather than with a separate key pair, and with the agent forwarding in section 3 it extends to `sudo` as well.
- **Renewal is not free.** A renewed certificate normally carries a new key pair, so the derived SSH key changes. Nothing needs editing on the server, but the thumbprint on the Windows side changes, and the command line you saved last year will stop working. Re-run the helper script to get the new one.

A quick way to confirm the server side before touching Windows, run on the Linux host:

```
sss_ssh_authorizedkeys someone@ad.example.com
```

If that prints one or more `ssh-rsa ...` lines, the server is ready and any remaining problem is on the client.

---

2. The Windows client, in short

Use **PuTTY-CAC**. It is the PuTTY fork that can authenticate with a certificate held in the Windows certificate store (`CAPI:`) or reached through a PKCS#11 module (`PKCS:`).

The command you want looks like this:

```
putty.exe -x509off -i CAPI:<sha1-thumbprint> -l someone@ad.example.com linux-host.ad.example.com
```

and, for scripted use:

```
plink.exe -x509off -batch -i CAPI:<sha1-thumbprint> -l someone@ad.example.com linux-host.ad.exam
ple.com
```

Three details in there are not obvious:

- `-x509off` is required. Section 4.2 explains why.
- `-l someone@ad.example.com` rather than `someone@ad.example.com@host`, because an Active Directory UPN contains an `@` and `user@host` parsing splits on it.
- `<sha1-thumbprint>` must be read **off the card**, not picked out of the Windows certificate store. Section 4.3 explains why.

Steps:

1. **Install PuTTY-CAC.** `winget install --id NoMoreFood.PuTTY-CAC`, or from `https://github.com/NoMoreFood/putty-cac/releases`. Stock PuTTY will not do; it has no `CAPI:` support at all.
2. **Optionally install OpenSC**, `winget install --id OpenSC.OpenSC`. It is only needed to read the thumbprint straight off the card as a cross-check; `certutil` can usually do that on its own.
3. **Insert the token** and check that Windows sees a card, not just a reader.
4. **Run the helper script** (section 5) to get the checks and the exact command line.
5. **Run the command it prints.** PuTTY will ask for the card PIN. Nothing in this project ever asks for, stores or logs a PIN.

Administrator rights are not needed for any of the checks. They are only needed if you end up installing a minidriver.

Minidrivers

A PIV token that Windows has already bound to a minidriver, and that is already provisioned with a certificate, needs nothing further. Windows ships an inbox PIV minidriver (`msclmd.inf`) that is enough to *use* the certificates on a card for authentication, signing and encryption.

A vendor minidriver matters for *provisioning and management* — enrolling a certificate onto the card, changing the card PIN, letting the MMC certificates snap-in write to the card. Installing one does not repair a stale certificate store entry, and is not a cure for every smart card error.

- **Yubico YubiKey** — YubiKey Smart Card Minidriver, `https://www.yubico.com/support/download/smart-card-drivers-tools/`, or `winget install --id Yubico.YubiKeySmartCardMinidriver`.
- **HID Crescendo** — HID Crescendo Minidriver. Installation documentation at `https://docs.hidglobal.com/crescendo-md-v1.3/MiniDriver/install_minidriver.htm`; HID's own documentation names `www.hidglobal.com/drivers` as the download location, which must be opened in a browser. There is no winget package.
- **Any other PIV token** — check that vendor's own support site. The helper script will still read the card and still emit a command line for an unrecognised vendor; it just will not pretend to know which driver that vendor ships.

---

3. sudo with the forwarded agent

`sudo` on the Linux host can authenticate against the smart card even though the card is plugged into the Windows client. `pam_ssh_agent_auth` verifies a signature from the forwarded SSH agent, so the card proves who you are a second time, at privilege-escalation time, with no password anywhere.

What the server already provides

Three pieces, all already configured and verified on the lab host.

In `/etc/pam.d/sudo`, inserted **before** `@include common-auth`:

```
auth    sufficient  pam_ssh_agent_auth.so authorized_keys_command=/usr/bin/sss_ssh_authorizedkeys
```

In `/etc/sudoers.d/91-ssh-agent-auth`:

```
Defaults env_keep += "SSH_AUTH_SOCK"
```

And the package `libpam-ssh-agent-auth` (on Ubuntu 24.04: noble/universe, 0.10.3-8).

The `env_keep` line is not optional. `sudo`'s `env_reset` strips `SSH_AUTH_SOCK` by default; the module then finds no agent, and because the line is `sufficient` rather than `required`, the stack **silently falls through to the password prompt**. Nothing reports an error. A sudo that keeps asking for a password is the expected symptom of forgetting that one line.

The `authorized_keys_command=` part is the architecturally important bit. The keys come from the same AD source `sshd` already uses, so **revoking the certificate removes sudo access too**. Most guides for this module tell you to paste a public key into `~/.ssh/authorized_keys` instead. Do not: a key pasted there survives revocation and silently undoes the single property that makes this whole design worth having.

Proven working, from the module's own debug output:

```
debug1: Running AuthorizedKeysCommand: "/usr/bin/sss_ssh_authorizedkeys" as "<user>" with argume
nt: "<user>"
matching key found: file/command /usr/bin/sss_ssh_authorizedkeys, line 3
Found matching RSA key: SHA256:<fingerprint>
ssh_rsa_verify: signature correct
Authenticated: `<user>' as `<user>'
```

The two things the client must get right

1. Pageant must be running, with the certificate loaded, at connect time. Agent forwarding is negotiated when the session opens. Loading the certificate into Pageant *after* the session is up does not retrofit it onto that session — you have to reconnect.

2. Agent forwarding must actually be on. Either pass `-A` on the command line, or enable it for the saved session. The per-session settings live here:

```
HKCU\Software\SimonTatham\PuTTY\Sessions\<session-name>
    AgentFwd   REG_DWORD   1      forward the agent for this session
    TryAgent   REG_DWORD   1      consult Pageant at all (default 1 if absent)
```

The equivalent of ticking *Connection* > *SSH* > *Auth* > *Allow agent forwarding* and re-saving:

```
reg add "HKCU\Software\SimonTatham\PuTTY\Sessions\my-linux-host" /v AgentFwd /t REG_DWORD /d 1 /
f
reg add "HKCU\Software\SimonTatham\PuTTY\Sessions\my-linux-host" /v TryAgent /t REG_DWORD /d 1 /
f
```

`-A` on the command line covers a single run regardless of `AgentFwd`; the saved value is what matters when the session is launched from the PuTTY UI.

The full command is the one from section 2 with `-A` added:

```
putty.exe -x509off -A -i CAPI:<sha1-thumbprint> -l someone@ad.example.com linux-host.ad.example.
com
```

What `-A` costs you

This is the one real price of the technique, and it is worth being blunt about.

While the session is alive, anyone with root on that server can use the forwarded agent to authenticate as you to other hosts. The agent signs whatever it is asked to sign, and root on the far end can reach the forwarded socket. It cannot extract the private key — that never leaves the card — but it does not need to: it can borrow your identity for as long as you stay connected.

So enable agent forwarding **per session**, for hosts you trust with your identity. Never as `Host *` in a client config, and not by default on jump hosts, which are exactly the machines most likely to be shared.

Verifying it

Inside the session:

```
ssh-add -l
```

must list the same key fingerprint the server accepts. Then:

```
sudo -k && sudo -l
```

No password prompt at all means the module authenticated: the `sufficient` line short-circuits the stack before `pam_unix` is ever reached. A prompt means it did not, and the stack fell through.

For diagnosis, add `debug` to the module options in `/etc/pam.d/sudo`. Remove it again afterwards — it logs about a dozen lines per `sudo` call.

Two measured red herrings

Both of these cost real debugging time. Neither is the problem.

`ssh-agent socket has incorrect permissions for owner` is not about permissions. It was produced by a malformed `SSH_AUTH_SOCK` value — the path had stray characters in it — and not by the socket's mode. Socket mode

`0755`, as `sshd` creates it, is **fine**; sudo authenticated successfully with exactly that. **Do not** `chmod` **the socket.** The useful diagnostic is:

```
printenv SSH_AUTH_SOCK
ls -l "$(printenv SSH_AUTH_SOCK)"
```

If the variable's value looks wrong, that is your answer.

`pam_ssh_agent_auth: Using default file=/etc/security/authorized_keys` appears in the debug output **even when** `authorized_keys_command` **is being used**. It reports the `file` option's default regardless of where the keys actually come from. The `Running AuthorizedKeysCommand:` line a few lines later is what tells you the command is in use. We nearly "fixed" a non-problem on the strength of that message.

---

4. Three traps

These three account for most of the time lost on this. Each produces a misleading symptom, and two of them produce no error message worth the name.

4.1 The Windows OpenSSH client cannot do this at all

`C:\WINDOWS\System32\OpenSSH\ssh.exe` — `OpenSSH_for_Windows_9.5p2` — **accepts the** `-I` **option and then silently ignores it.** PKCS#11 support is compiled out of that build. There is no warning, no error and no non-zero exit to tell you so.

What you see:

```
ssh -I "C:\Program Files\OpenSC Project\OpenSC\pkcs11\opensc-pkcs11.dll" someone@ad.example.com@
linux-host.ad.example.com
```

The option is accepted. The card is never touched. Authentication proceeds as though no key had been offered and then fails for want of one.

How to confirm it rather than guess: run the same command with `-vvv`. A build with PKCS#11 support logs the provider being loaded. This one prints **no provider line and no** `dlopen` **line at all** — the absence is the diagnosis.

Also ruled out: the `ssh.exe` that ships with Git for Windows (10.5p1) does implement PKCS#11, but loading a native `opensc-pkcs11.dll` into that MSYS2 binary crashes reproducibly. Not a workaround.

So: use PuTTY-CAC. Do not spend an afternoon on `ssh -I`.

4.2 X.509 mode makes the server refuse the key

PuTTY-CAC can offer `x509v3-ssh-rsa` (RFC 6187) when its X.509 mode is on. **OpenSSH does not implement that algorithm.** The server refuses the key, and logs

```
unsupported public key algorithm
```

The client does not show you that. All you get is

```
Server refused our key
```

which sends people off to check `authorized_keys`, SSSD mapping rules and account permissions — none of which is the problem.

The fix is `-x509off` on every invocation. Note that X.509 mode is a **persistent, global PuTTY-CAC setting, shared with Pageant**: once something turns it on it stays on for later runs and for other tools in the suite, which is exactly why passing `-x509off` explicitly every time is worth the eight characters.

4.3 The thumbprint must come from the card

This is the subtle one, and the check most worth running.

A certificate entry in the Windows store (`Cert:\CurrentUser\My`) names a key container. That container may no longer exist — the card it belonged to has been reprovisioned, replaced, or enrolled again under a new container. The store entry survives; the key behind it does not.

Pick such an entry and Windows tells you:

```
the smart card cannot perform the requested operation or the operation
requires a different smart card
```

which is both accurate and useless, because it does not say *which* card, and the certificate looks perfectly valid in the MMC snap-in.

Stale entries accumulate quickly. A workstation that has enrolled a handful of cards can easily hold more dead entries than live ones, with several of them sharing one subject name — so picking "the one with my name on it" is a coin toss.

The reliable test is to compare the two sources directly. What the card actually holds:

```
certutil -scinfo -silent
```

```
Key Container = 00000000-1111-2222-3333-444444444444 [Default Container]
Cert Hash(sha1): aaaaaaaabbbbbbbbccccccccddddddddeeeeeeee
```

What the store entry claims:

```
certutil -silent -user -store My aaaaaaaabbbbbbbbccccccccddddddddeeeeeeee
```

```
Key Container = 00000000-1111-2222-3333-444444444444
Provider = Microsoft Smart Card Key Storage Provider
```

If the two `Key Container` GUIDs differ, that certificate is unusable, whatever the store says about it. A store entry whose container cannot be opened at all reports `Missing stored keyset` instead, which is the same problem in a louder voice.

Note that `certutil -scinfo` returns a non-zero exit code when *any* card in *any* reader has a problem, while still printing complete, correct output for the healthy ones. Do not treat its exit code as "no data".

`-silent` keeps `certutil` from putting up a PIN prompt. Reading certificates and container names needs no PIN; only the eventual SSH login does.

---

5. The helper script

`scripts/windows/New-SmartCardSshCommand.ps1` runs the checks above and prints the command. It is read-only: it never launches PuTTY, never downloads anything, never installs a driver, never reads or writes a private key, and never asks for, stores or logs a PIN. It needs PowerShell 5.1 or 7, and no administrator rights.

```
.\scripts\windows\New-SmartCardSshCommand.ps1 `
    -User someone@ad.example.com `
    -ComputerName linux-host.ad.example.com
```

It reports each prerequisite as PASS / WARN / FAIL with a one-line remedy:

Check	What it means
PuTTY-CAC present	Found, and distinguished from stock PuTTY
OpenSC present	Optional; only to read the thumbprint off the card
Reader and card present	A card, not merely a reader
Key container matches the card	Trap 4.3 — the check that matters
Pageant is running and serving	With `-ForwardAgent`; detected by its named pipe
Saved session has agent forwarding	With `-ForwardAgent -SessionName`; reads `AgentFwd`/`TryAgent`

It then identifies the token family (Yubico YubiKey, HID Crescendo, or a generic PIV token) and the minidriver Windows has bound to it, and emits both the `putty.exe` and the `plink.exe` command lines.

Useful parameters:

Parameter	Purpose
`-User`	Login name sent with `-l`, e.g. `someone@ad.example.com`

Parameter	Purpose
`-ComputerName`	SSH server, e.g. `linux-host.ad.example.com`
`-Port`	Non-default SSH port
`-PuttyPath`	Point at a specific `putty.exe`
`-Thumbprint`	Use this certificate instead of choosing from the card
`-Reader`	Restrict to readers whose name contains this text
`-UseOpenSC`	Also read the thumbprint via PKCS#11, as a cross-check
`-ForwardAgent`	Add `-A`, and check Pageant and the session's agent settings (section 3)
`-SessionName`	Saved PuTTY session whose `AgentFwd`/`TryAgent` to report
`-Verbose`	Show the external commands being run

With no `-User` or `-ComputerName` it still runs every check and shows the command shape with example values, which makes it usable purely as a diagnostic.

More than one token attached is a normal state and the script handles it without guessing. It lists what it found and stops, asking you to choose with `-Reader` or `-Thumbprint`. It will not silently pick one for you.

Exit codes: `0` a command was emitted, `1` no usable certificate, `2` the choice was ambiguous.

The script is read-only by nature, so there is nothing for `-WhatIf` to suppress.

---

6. If it still fails

In rough order of likelihood:

- `Server refused our key` — add `-x509off` (4.2).
- `the smart card cannot perform the requested operation` — wrong store entry; re-read the thumbprint from the card (4.3).
- The card PIN prompt never appears — the certificate was not found in the store at all. Re-insert the token so Windows registers it, or address the card directly with `-i PKCS:<id>=<path-to-opensc-pkcs11.dll>`.
- Authentication fails with no card activity and you are using `ssh.exe` — see 4.1; that client cannot do it.
- The key is offered and refused, and `-x509off` is already set — check the server: run `sss_ssh_authorizedkeys someone@ad.example.com` on the host and confirm it returns a key, then compare it against the public key in the certificate on the card.
- Everything passes but the account is rejected after the key is accepted — that is authorisation, not authentication. Look at SSSD's access control and `local_auth_policy`, not at the card.
- SSH works but `sudo` still asks for a password — the `sufficient` line fell through. Check `env_keep += "SSH_AUTH_SOCK"` first, then that Pageant was running with the certificate loaded *before* you connected, then that `-A` was actually passed. See section 3, and do not chase the socket-permissions message.

One note on PKCS#11 libraries, since it costs people time: a DLL with `pkcs11` in its name is not necessarily a PKCS#11 module. At least one vendor ships a `...-pkcs11....dll` that exports no `C_*` functions at all, so `C_GetFunctionList` cannot be resolved and any PKCS#11 consumer fails to load it. If `-i PKCS:...` fails to initialise, verify the module exports `C_GetFunctionList` before assuming the card or the client is at fault. The OpenSC module (`opensc-pkcs11.dll`) is the known-good one for PIV.